UNIVERSITY REGISTRATION AND ONLINE PAYMENT SYSTEM

Middleware Architecture, Integration Testing Plan, and Performance Risks

This report describes the proposed architecture of a University Registration and Online Payment System, the integration testing approach, and the main performance and implementation risks. It presents a proposed design for the assignment, not a report of a deployed university system.

DELIVERABLE 1: MIDDLEWARE ARCHITECTURE AND COMPONENTS

1.1 System Overview

Our chosen system is the University Registration and Online Payment System. It is a web-based platform designed to connect the existing student records system, course scheduling services, and a third-party payment processor. Students can access their accounts, check course prerequisites and available classes, view tuition fees, and make enrollment payments through one portal. The main components are the Student Web Portal, where students submit registration requests and view their status; the API Gateway, which receives and routes requests; the Registration Service, which coordinates registration and course validation; the Course Catalog Microservice, which manages course information, prerequisites, schedules, and available slots; the Payment Integration System, which creates payment requests and verifies payment results; the Legacy Student Records System, which stores student and enrollment records; and the external Payment Gateway, such as Stripe or PayPal, which processes the payment.

1.2 Middleware Mapping

The main communication problem is that the newer registration components must exchange information with the older student records system and an external payment provider. These components may use different data formats, and the legacy system may not always be available when a payment has already been confirmed. The design therefore needs clear interfaces between components and a safe way to recover an enrollment update without charging the student again.

Middleware categories that apply

Communication Middleware: REST APIs over HTTPS allow the portal, API Gateway, registration components, and compatible external interfaces to exchange requests and responses.

Data Management Middleware: A student-records API or adapter maps the newer system's requests to the data and operations supported by the legacy student records system.

Distribution Middleware: The API Gateway routes requests to the appropriate component, such as registration or course validation, rather than requiring the portal to communicate separately with every service.

Application Co-operation Middleware: The Payment Integration System coordinates the verified payment result with the enrollment update. If the legacy system is unavailable, the update is recorded as pending for retry or staff reconciliation.

System Management Middleware: Logs, status tracking, monitoring, and error handling help the team identify failed requests and follow up on pending enrollment updates.

Object Management Middleware is not a primary requirement in this design because the proposed components communicate through APIs rather than relying on distributed remote objects.

1.3 Technology Selection

Technology 1: REST APIs (with RPC as an alternative)

We propose REST APIs as the main way for the system components to communicate. For example, the API Gateway can send a registration request to the Registration Service, while the Payment Integration System can communicate with the payment provider and the student-records adapter through defined interfaces. REST APIs commonly use HTTP and JSON, making it possible for components built with different technologies to exchange information. RPC is a related but different approach to remote communication; it could be used instead if the system were designed around remote procedure calls.

Technology 2: Open Database Connectivity (ODBC)

ODBC may be used if the legacy database supports a suitable ODBC driver and the university authorizes direct database access. It provides a standard interface through which compatible applications can query or update database records. However, ODBC alone does not prevent conflicting updates or replace the need for validation, permissions, and transaction handling. If the legacy system already provides a supported API, using that API may be safer than connecting directly to its database.

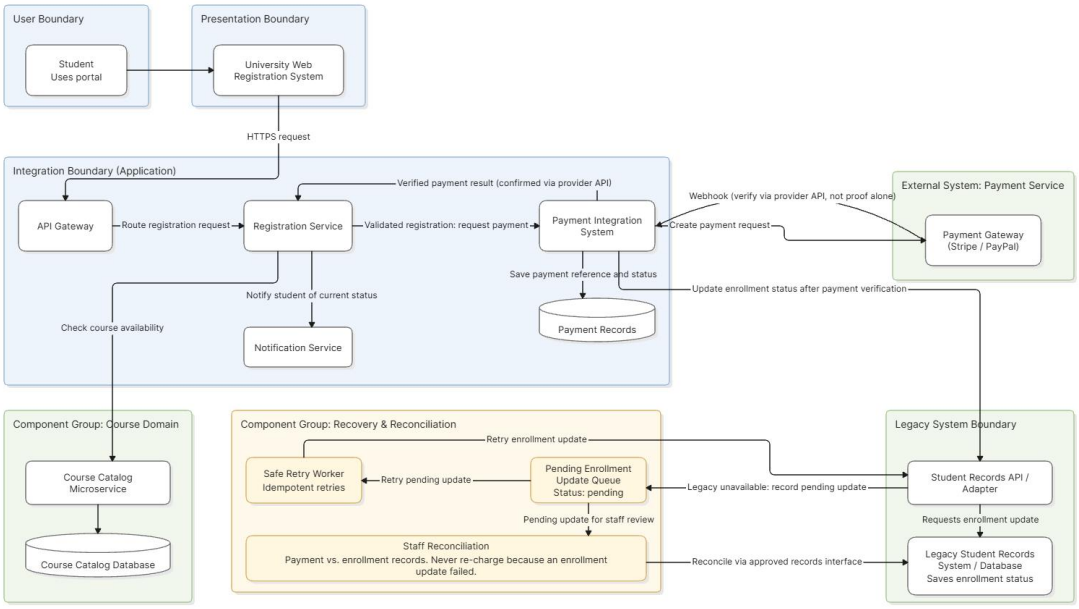


Figure 1. System Architecture of the University Registration and Online Payment System

Figure 1 presents the main system components and their communication paths. It should make clear that payment confirmation is handled by the Payment Integration System and that enrollment status is updated through the legacy student-records interface.

DELIVERABLE 2: INTEGRATION TESTING PLAN

2.1 Integration Testing Strategy: Continuous Testing

We would use Continuous Testing for the University Registration and Online Payment System. Instead of waiting until all components are finished, the team would run automated component, integration, and regression tests as changes are made. This suits the system because the portal, API Gateway, Registration Service, Course Catalog Microservice, Payment Integration System, and legacy student-records interface must work together. Testing throughout development can reveal interface errors early and help the team check that a change to one component has not broken an existing feature. It requires test automation and coordination, but it gives the team regular feedback as the system develops.

2.2 System Layers and Test Components

A test driver is a temporary program that calls the component being tested when the normal calling component is not ready. For example, a driver can send sample registration requests to the API Gateway or call the student-records adapter directly. A stub temporarily imitates a component that the tested component depends on. For example, a payment gateway stub can return simulated successful, declined, pending, or temporary-failure responses.

Use a driver when the component that normally makes the call is unavailable. Use a stub when the component that normally receives the call is unavailable or when a controlled response is

needed. Replace test drivers and stubs with real components for end-to-end tests where the actual integration environment is available.

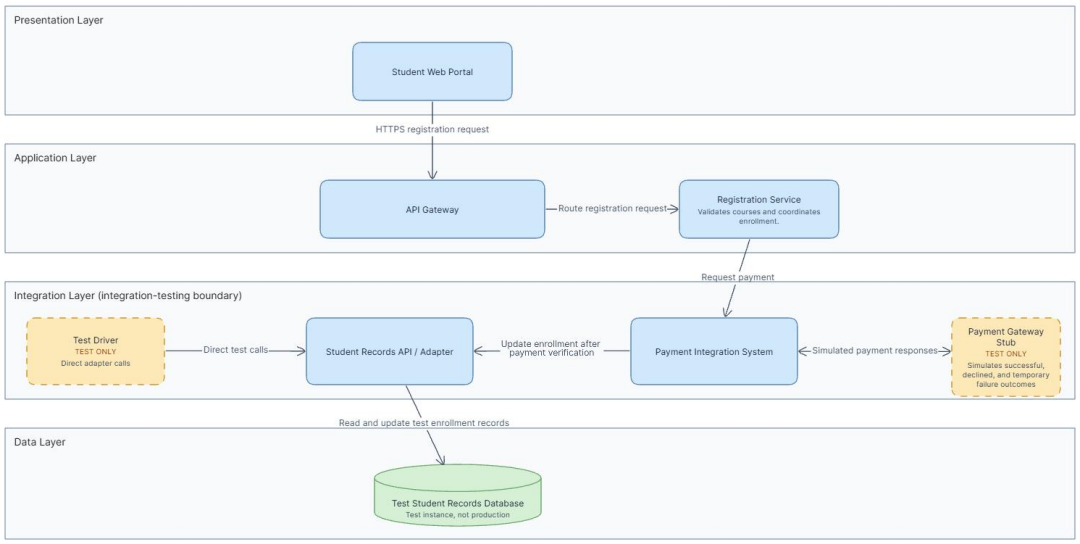


Figure 2. System Layers and Test Components for Continuous Testing

2.3 Integration Test Cases

Test Case IT-01: Student Verification

Components: API Gateway and Student Records API/Adapter.

Objective: Check that a registration request is accepted only when the student ID is valid.

Steps: Send one registration request with a valid student ID and another with an invalid ID through the API Gateway. Check the response from the student-records interface.

Expected result: The valid student can proceed to course validation. The invalid student is denied further registration and receives an appropriate error message.

Test Case IT-02: Course Availability and Prerequisites

Components: Registration Service and Course Catalog Microservice.

Objective: Check that course availability and prerequisites are validated before enrollment proceeds.

Steps: Submit a registration request for a course with available slots and satisfied prerequisites. Repeat with a course whose prerequisites are not met or whose slots are full.

Expected result: The eligible request proceeds to the next enrollment step. The ineligible request is rejected with a clear explanation.

Test Case IT-03: Payment Confirmation and Enrollment Update

Components: Payment Integration System, Payment Gateway (or stub), and Student Records API/Adapter.

Objective: Check that a verified payment updates enrollment correctly and that a failed legacy update does not cause another charge.

Steps: Simulate a successful payment and verify its status using the provider's supported verification method. Then make the legacy student-records interface unavailable during the enrollment update. Restore the interface and run the retry process.

Expected result: The confirmed payment reference is stored once. While the legacy update is unavailable, the enrollment update is marked pending. A safe retry reuses the original payment reference and does not initiate another payment. Once the legacy update succeeds, the enrollment status is updated and the student is notified.

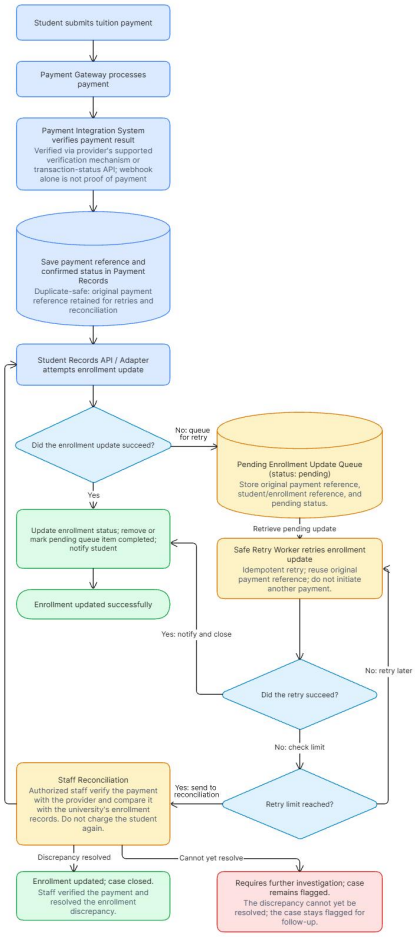


Figure 3. Payment Confirmation and Enrollment Recovery Workflow

DELIVERABLE 3: PERFORMANCE RISKS, COSTS, AND DISADVANTAGES

3.1 Potential Performance Bottleneck and Testing

A possible bottleneck is the API Gateway and the request path behind it during peak enrollment. Registration requests may require student-record verification, course availability checks, and communication with the payment integration components. If requests wait too long for a slow

legacy system or external payment provider, response times can increase and requests may accumulate.

The team can use a load-testing tool such as Apache JMeter or k6 to simulate many concurrent students and measure response time, throughput, error rate, and resource usage. A test driver can generate registration requests, while a payment gateway stub can deliberately return slow responses or temporary failures. The team should define an acceptable response-time and error-rate target before testing. The system should fail gracefully when capacity is reached, return a clear error or pending status, and avoid creating duplicate payments or enrollment records. The test should also confirm that a slow external provider does not leave requests waiting indefinitely.

3.2 Costs and Disadvantages

Complex Testing Setup

The team must create and maintain test drivers and stubs that represent different responses from the payment provider and legacy system. This takes time, and the test doubles must be updated when interfaces change. They also cannot fully reproduce every behavior of the real external services, so final tests in an authorized sandbox or test environment are still needed.

Dependency on Third-Party Services

The system relies on the external payment provider for payment processing and status verification. If its sandbox is unavailable, rate-limited, or changed, some integration tests may be delayed. The team can reduce this impact by using stubs for routine tests, checking the provider's documentation, and reserving sandbox tests for cases that require real provider behavior.

Legacy Compatibility and Recovery Work

The legacy student-records system may have older interfaces or data formats that do not match the new services. An adapter, mapping rules, and extra validation may be needed. The system also needs monitoring, retry limits, and staff reconciliation procedures for cases where payment is confirmed but the enrollment update remains unresolved. These controls add implementation and maintenance work, but they help prevent duplicate charges and inconsistent enrollment records.